CraftBot: An LLM Interlocutor in a Code-First Approach to Architectural Design

First author
First institution
First address
firstemail@mail.com

Second author
Second institution
Second address
secondemail@mail.com

Abstract

This paper investigates whether large language models can participate meaningfully in architectural design when constrained to operate through executable CAD code. We introduce CraftBot, an AI system that generates architectural geometry in Blender via Python scripts, forcing all design reasoning to manifest as procedural construction. Thirteen experiments explore how CraftBot interprets heterogeneous reference material, including photographs, drawings, and construction manuals, and how it incrementally refines geometry through an iterative loop of code generation, execution, visual feedback, and revision. The experiments show that LLMs can assemble coherent architectural structures, absorb domain-specific construction logic, and maintain parametric relationships over multiple iterations when code becomes the primary representational medium. At the same time, CraftBot remains largely incapable of autonomously detecting its own geometric errors, requiring human guidance to identify inconsistencies and resolve ambiguities. The paper argues that architectural relevance for AI emerges from integration of the real-world knowledge in expert domains and its participation in the design process through a constrained, executable construction logic.

Introduction

In recent years, large language models (LLMs) have been increasingly used as engines for *3D generation*. In computer graphics, the term has come to describe pipelines that transform text prompts into visually convincing meshes or implicit fields. In architecture, however, 3D models do not primarily function as images. They are instruments of reasoning, coordination, and construction. They encode dimensions, tolerances, material hierarchies, and assembly logic. They are expected to remain editable under change, to carry semantic structure, and to survive translation between design, engineering, and fabrication. To speak of 3D generation in an architectural context therefore means something more demanding: the generation of procedural, parametric, and interpretable objects that can be used in real design processes.

We argue that if LLMs are to become relevant for architectural design, they must be forced into already established procedural workflows and produce geometry by building it.

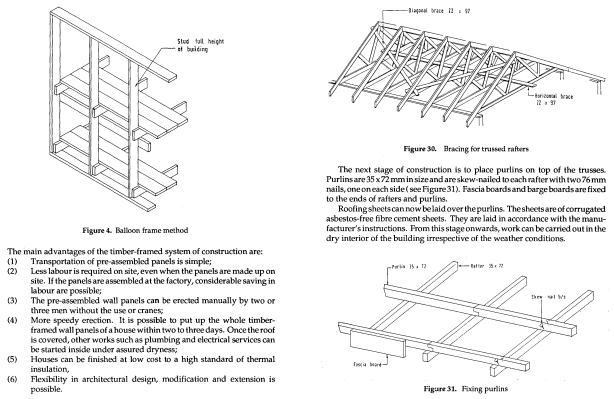

Figure 4. Balloon frame method

The main advantages of the timber-framed system of construction are:
(1) Transportation of pre-assembled panels is simple;
(2) Less labour is required on site, even when the panels are made up on site. If the panels are assembled at the factory, considerable saving in labour are possible;
(3) The pre-assembled wall panels can be erected manually by two or three men without the use or cranes;
(4) More speedy erection. It is possible to put up the whole timber-framed wall panels of a house within two to three days. Once the roof is covered, other works such as plumbing and electrical services can be started inside under assured dryness;
(5) Houses can be finished at low cost to a high standard of thermal insulation,
(6) Flexibility in architectural design, modification and extension is possible.

Figure 30. Bracing for trussed rafters

The next stage of construction is to place purlins on top of the trusses. Purlins are 35 x 72 mm in size and are skew-nailed to each rafter with two 76 mm nails, one on each side (see Figure 31). Fascia boards and barge boards are fixed to the ends of rafters and purlins.

Roofing sheets can now be laid over the purlins. The sheets are of corrugated asbestos-free fibre cement sheets. They are laid in accordance with the manufacturer's instructions. From this stage onwards, work can be carried out in the dry interior of the building irrespective of the weather conditions.

Figure 31. Fixing purlins

Figure 1: Pages from the *Construction Manual of Prefabricated Timber House* (Midon et al. 1996), reference manual used as an input for *Experiment 04.*

On this basis, we introduce *CraftBot*, a system that generates architectural models exclusively through Python code executed in a CAD environment. CraftBot does not output meshes, sketches, or symbolic abstractions. It outputs instructions that construct geometry in the same way a computational designer would: by placing elements, defining relationships, and maintaining parametric consistency. We hypothesize that LLM-based 3D generation becomes architecturally relevant only when executable code becomes the primary representational medium. Code allows parametric control, preserves design intent, supports reproducibility, and forces geometric accountability. It also enables a form of iterative reasoning in which outputs can be tested, visualized, critiqued, quantified and evaluated.

We implement CraftBot in Blender, a free and open-source 3D modeling software that exposes a complete and transparent geometry API through Python programming language. While Blender is not a traditional architectural CAD platform in the same sense as Rhino or Revit, it provides equivalent capabilities for procedural geometry generation and visual validation. The approach presented in this paper is not Blender-specific and can be transferred to any CAD environment that admits procedural access to its geometry kernel.

State Of The Art

Research on LLM-based 3D generation is often presented as a rapidly converging field, but from the perspective of architectural design it remains deeply fractured. Different communities speak about *generation* while referring to non-interchangeable paradigms: images that resemble geometry, meshes that suggest volume, or parametric sequences that encode construction. These embody different assumptions about what a model is supposed to do and what kind of intelligence is being delegated to the machine. For the purposes of architectural design and CAD-oriented workflows, four tendencies matter most:

- **Image-driven text-to-3D synthesis** - examples are *DreamFusion* which uses pretrained text-to-image diffusion models to guide the optimization of implicit 3D representations (Poole et al. 2022), or their extensions like *Magic3D* (Lin et al. 2023). Surveys such as (Jiang 2024) show how systematically this line of work has developed.
- **Code-driven CAD program synthesis** - for example *LLMto3D's* multi-agent structure decomposes language into objects and relations, translates these into parametric modeling code, and assembles the result within a CAD environment (El Hizmi et al. 2024). The central promise of this paradigm is *editability*, but the survey by (Zhang et al. 2025) shows persistent issues like execution brittleness, ambiguity resolution and tool dependence.
- **Computational co-creativity frameworks** - for instance, in the framing of co-creative systems within traditions of human–computer interaction in the works of (Karimi et al. 2018), or mixed-initiative models following (Yannakakis, Liapis, and Alexopoulos 2014).

Methodology

Our initial idea for implementing CraftBot was directly inspired by the AI Chair project by the artist James Bridle where he tasked Mistral AI to describe a process of constructing a functional chair based on the list of provided scrap materials. The artist then proceeded to construct the chair by following the written instructions, often taking liberty in interpreting them and correcting for geometric or constructive inconsistencies (Bridle 2024). Bridle's experiment suggested that LLMs possess a latent capacity to reason about space, structure, and assembly, even if that reasoning is not geometrically precise.

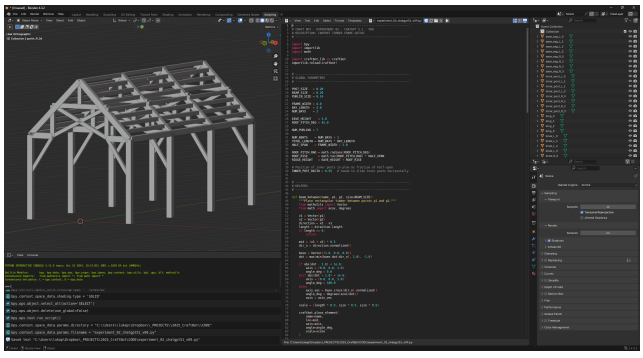

Figure 2: Blender interface for *Experiment 02* - generated model on the left, Python code in the middle with the Python console bottom left, list of geometry elements and scene configuration on the right.

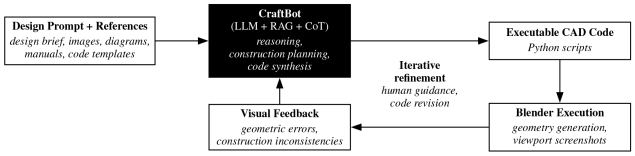

Figure 3: CraftBot procedural refinement loop.

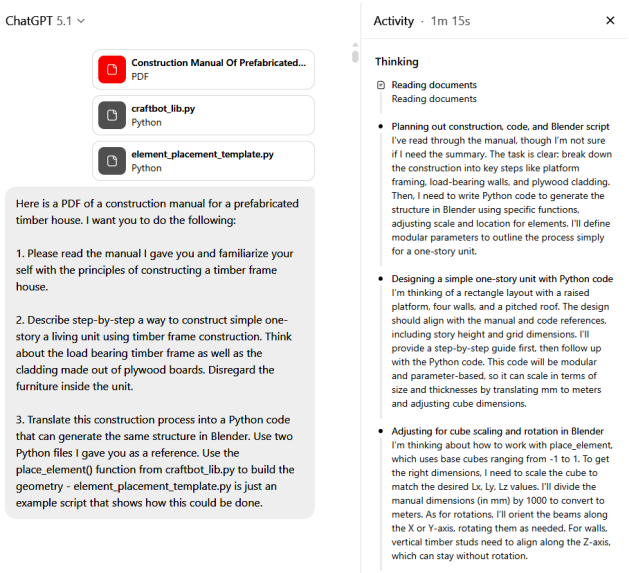

Figure 4: CraftBot chat interface showing the start prompt and input references for *Experiment 04* (left), and a crop of Chain-Of-Thought steps that are created before the final answer is delivered (right).

Our question was whether this capacity could be forced into a more rigorous form by combining two techniques that deliberately constrain the LLM: Retrieval-Augmented Generation (RAG) and Chain-Of-Thought (CoT). RAG allowed us to inject domain-specific construction knowledge directly into the prompt context, grounding the model's responses in expert material rather than statistical plausibility (Lewis et al. 2020). CoT, in turn, forced the model to externalize intermediate reasoning steps, slowing down its responses and making failures of logic or consistency visible (Wei et al. 2022).

Instead of asking the LLM to explain how something might be built, we required it to write Python code that actually builds it inside Blender as the execution environment. Each experiment began with a prompt that framed a design task in deliberately simple terms and supplied reference material in one of three forms: visual (photographs, diagrams, plans), textual (manuals, descriptions, technical literature),

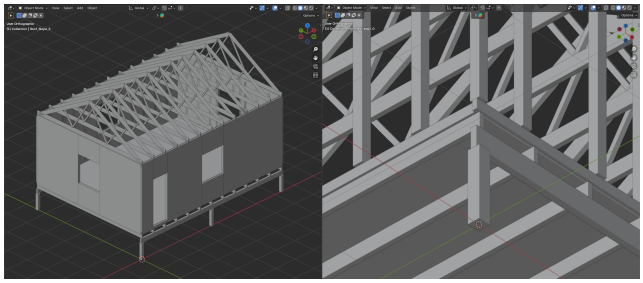

Figure 5: Blender model generated with the Python code output of *Experiment 04*.

and procedural (Python modules, geometry templates). Every CraftBot response included executable Python code. There were no intermediate representations, no sketches, and no symbolic abstractions. Code was the sole medium of commitment. If the model misunderstood something, the error would appear as intersecting solids, missing structure, or broken spatial logic.

After each code execution, Blender screenshots were fed back to CraftBot. The model was asked to analyze its own output, compare it to the original brief and references, and propose improvements that were then implemented through new code. This established an iterative loop in which language, vision, and procedure were forced into conversation. The loop was not fully autonomous. CraftBot often failed to identify its own mistakes without guidance. Human intervention took the form of clarifying geometric errors, pointing to inconsistencies, or annotating screenshots. There was no formal stopping criterion. Iterations were terminated when improvements became marginal or when the model's corrections began to oscillate rather than converge.

All experiments were conducted between November 2025 and January 2026 using ChatGPT 5.1 which natively supported both RAG and CoT. For reasons of transparency and reproducibility, all prompts, code outputs, screenshots, and 3D models are archived in a public GitHub repository. The paper itself only references them selectively.

Experiments

The thirteen experiments were organized pragmatically, according to the kind of reference material that anchored the initial prompt: (1) purely visual sources, and (2) hybrid sources that combined images with textual or technical documentation. In all experiments, code-based references were present in the form of Python geometry functions and example templates.

Input type – visual references

Experiments 01 and 02 were conceived as stress tests of minimal input. Could a handful of images be sufficient to trigger a coherent procedural reconstruction? Both experiments relied on two rendered views of timber-frame carports taken from a company specializing in traditional timber construction in British Columbia (Timber Frame HQ 2025). Experiment 01 targeted a structure with a king post truss, while Experiment 02 focused on a slightly more complex structure with a hammer beam truss. In first outputs, posts, beams, rafters, and truss members appeared in roughly correct locations. CraftBot could assemble a plausible structure, yet it was almost blind to its own geometric errors. We began correcting CraftBot in the same way one would correct a student: naming the elements, pointing to relationships, sometimes drawing directly onto screenshots. Once the errors were linguistically grounded in the model's own vocabulary, CraftBot could respond. After several iterations, both truss systems converged to structurally coherent configurations.

Experiment 03 raised the stakes by shifting from abstract timber structures to a specific architectural object: *The VIPP Shelter* by Morten Bo Jensen and Arcgency (Bo Jensen 2015). Here, the references included plan drawings and photographs. CraftBot reconstructed the basic volume, skylights, internal partitions, floor joists, and wall articulation. We stopped short of full detailing because the available references did not justify deeper constructive claims.

Experiment 06 returned to historical precedent with Jean Prouvé's *6x6 Demountable House* (Prouvé 1944). Six assembly diagrams and construction photographs were sufficient for CraftBot to produce a remarkably complete initial model: columns, slanted roof, portal beam, purlins, wall and roof panels, and door opening appeared already in the very first iteration.

Input type – hybrid references (images + text)

Experiment 04 marked a methodological turning point. Instead of sparse visual cues, CraftBot was given a complete technical manual: the *Construction Manual of Prefabricated Timber House*, a 60-page PDF (Midon et al. 1996). The manual contains drawings, sections, elevations, and explanatory text, and the CraftBot had to navigate all of it. We allowed the iteration loop to run until a fully articulated building emerged: foundation posts, floor structure, wall studs, interior and exterior boards, door and window frames, and a gable roof with trusses. Fourteen iterations were required. The resulting model was not perfect, but it was internally coherent, and we used it as a template for some of the subsequent experiments.

Experiment 07 deliberately broke architectural orthodoxy. Using the codebase from **Experiment 04** and reference images of Frank Gehry's two projects - *Gehry Residence* and the *Schnabel House* (Gehry 1978; 1989), CraftBot was asked to *deconstruct* the existing structure. Initially it failed in a banal way, scattering random volumes. Only when we reframed deconstruction as a transformation of planar surfaces into ruled surfaces did the model begin to behave architecturally. The resulting building employed ruled surfaces clad with flat panels, echoing real construction strategies.

Experiment 08 extended the hybrid-document approach using *The Segal Method*, a 27-page PDF (Broome 1986a; 1986b). CraftBot produced a two-story house with staircase, double-height space, mixed roof types, and multiple façade systems. Fourteen iterations were required to stabilize geometry, naming conventions, and Blender organization. At

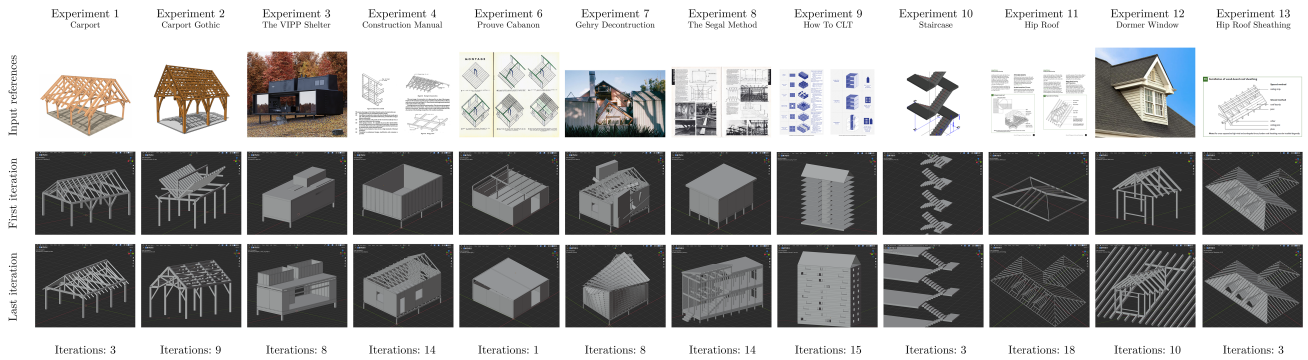

Figure 6: Overview table with outputs of all experiments.

this point CraftBot began to exhibit something resembling architectural consistency.

Experiment 09 tested scale. Using the *How to CLT Handbook*, a 38-page PDF (Bahari et al. 2024), CraftBot generated a ten-story building with core, apartments, elevators, and roof. The staircase proved to be the hardest element, leading to **Experiment 10**, which isolated it as a design problem. Once solved, the staircase code was merged back into the larger building.

Experiments 11, 12, and 13 pushed toward roof complexity using *Canadian Wood-Frame House Construction*, a 31-page PDF (Canada Mortgage and Housing Corporation 2013). Together they produced a multi-wing hip roof with dormers, aligned rafters, and continuous sheathing. Thirty-one iterations were required. This was the most fragile and demanding structure generated in the paper.

Conclusion

This paper began with a relatively simple question: can a LLM participate meaningfully in architectural design if it is forced to operate through code rather than through images or descriptive language? The experiments with CraftBot suggest that the answer is yes, but not in the way common narratives around *AI design* might imply.

Working through Python code in Blender imposed a discipline on the system: geometry had to be constructed, rather than imagined; errors had to be embodied in coordinates and intersections, rather than hidden behind visual plausibility. This made both the strengths and weaknesses of CraftBot unusually transparent. The system could absorb architectural knowledge from manuals and diagrams, replicate construction logic, and maintain parametric relationships over many iterations. At the same time, it repeatedly failed to recognize its own geometric inconsistencies without external prompting. CraftBot behaved as a procedural interlocutor whose proposals had to be tested, corrected, and sometimes resisted.

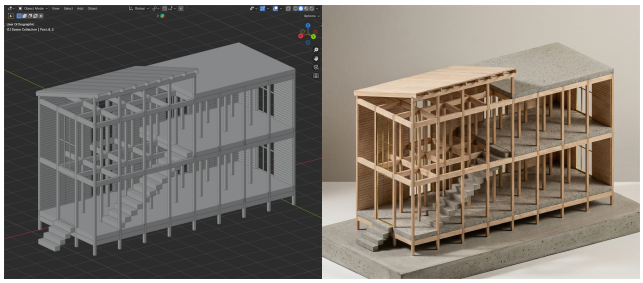

Figure 7: Blender model generated with the Python code output of *Experiment 08* (left), and its physical model visualization (right).

Contributions of the paper are threefold:

- We show that LLMs can operate meaningfully within a **code-first modeling paradigm**, producing architectural geometry that remains parametric and inspectable.
- We demonstrate that **iterative refinement** based on visual feedback and code reuse can form a self-improving design loop, even if that loop remains partially dependent on human guidance.
- We provide a set of concrete experiments in which historical and contemporary building typologies are reconstructed from heterogeneous sources, demonstrating how **real-world knowledge in expert domains** can be integrated into such a design process.

We see this paper as an initial foray into the topic. Future work should include at least the following quantitative criteria: a table showing number and type of errors made, a human expert assigned score for each design iteration, a quantitative mesh model similarity score for meshes from different iterations, systematic comparison of input modalities etc.

Further explorations could introduce explicit geometric reasoning into the process: collision detection, constraint solving, dimensional validation. Our hypothesis is that adding such mechanisms will change the character of the system where it will begin to argue with itself, producing a more rigid, and perhaps less inventive, form of procedural intelligence. Future work should lean into this instability and create LLM-based design systems that become spaces where intent, code, and geometry remain in continuous negotiation.

Acknowledgments

This section will be added after the paper review stage.

References

Bahari, N.; Elangovian, E.; Moattar, K.; Orth, F.; Pettersson Sandmark, N.; Sarrafzadeh, S.; and Tjernberg, F. 2024. Architectural Guidelines for Early Stages: How to CLT Handbook (2nd edition). https://arkemi.se/images/downloads/Arkemi_How-to-CLT_Handbook_30.pdf. Accessed: 21 November 2025.

Bo Jensen, M. 2015. The vipp shelter / vipp. https://vipp.com/en/world-of-vipp/our-guesthouses/vipp-shelter. Accessed: 21 November 2025.

Bridle, J. 2024. Ai chair 1.1: Llamafiles and mistral. https://booktwo.org/notebook/ai-chair-1-1-llamafiles-and-mistral/. Accessed: 21 November 2025.

Broome, J. 1986a. The segal method. *Architects' Journal* 31–68. Special issue on The Segal Method, London.

Broome, J. 1986b. The segal method. https://ia801302.us.archive.org/12/items/THESEGALMETHOD/THE_SEGAL_METHOD_text.pdf. PDF of original article from Architects' Journal (5 November 1986), accessed 21 November 2025.

Canada Mortgage and Housing Corporation. 2013. Canadian wood-frame house construction : NH17-3/2013E-PDF. https://publications.gc.ca/collections/collection_2014/schl-cmhc/NH17-3-2013-eng.pdf. Accessed: 21 November 2025; Government of Canada digital publication.

El Hizmi, B.; Shkolnik, A.; Austern, G.; and Sterman, Y. 2024. Llmto3d - generating parametric objects from text prompts. In Nahmad-Vazquez, A.; Johnson, J.; Taron, J.; Rhee, J.; and Hapton, D., eds., *ACADIA 2024*, ACADIA 2024: Designing Change - Proceedings Volume 1 for the 2024 Association for Computer Aided Design in Architecture Conference, 157–166.

Gehry, F. 1978. Gehry residence. https://en.wikipedia.org/wiki/Gehry_Residence. Accessed: 21 November 2025.

Gehry, F. 1989. Schnabel house. https://www.beautiful-houses.net/2013/02/frank-gehrys-beautiful-architecture.html. Accessed: 21 November 2025.

Jiang, C. 2024. A survey on text-to-3d contents generation in the wild. *arXiv preprint arXiv:2405.09431*.

Karimi, P.; Grace, K.; Maher, M. L.; and Davis, N. 2018. Evaluating creativity in computational co-creative systems.

Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In Larochelle, H.; Ranzato, M.; Hadsell, R.; Balcan, M.; and Lin, H., eds., *Advances in Neural Information Processing Systems*, volume 33, 9459–9474. Curran Associates, Inc.

Lin, C.-H.; Gao, J.; Tang, L.; Takikawa, T.; Zeng, X.; Huang, X.; Kreis, K.; Fidler, S.; Liu, M.-Y.; and Lin, T.-Y. 2023. Magic3d: High-resolution text-to-3d content creation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Midon, M. S.; Pun, C. Y.; Tahir, H. M.; Kasby, N. A. M.; et al. 1996. Construction manual of prefabricated timber house. *Forest Research Institute Malaysia* 14.

Poole, B.; Jain, A.; Barron, J. T.; and Mildenhall, B. 2022. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv*.

Prouvé, J. 1944. 6x6 demountable house. https://www.jeanprouve.com/en/fiche/1944-6. Accessed: 21 November 2025.

Timber Frame HQ. 2025. Timber frame plans. https://timberframehq.com/timber-frame-plans/. Accessed: 21 November 2025.

Vaidhyanathan, V.; T R, R.; and Garcia del Castillo Lopez, J. L. 2023. Spacify: A generative framework for spatial comprehension, articulation and visualization using large language models (llms) and extended reality (xr). ACADIA 2023: Habits of the Anthropocene: Scarcity and Abundance in a Post-Material Economy - Proceedings Volume 2 for the 2023 Association for Computer Aided Design in Architecture Conferencee, 430–443.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; Zhou, D.; et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35:24824–24837.

Yannakakis, G. N.; Liapis, A.; and Alexopoulos, C. 2014. Mixed-initiative co-creativity. In *Proceedings of the 9th International Conference on the Foundations of Digital Games (FDG)*.

Zhang, L.; Le, B.; Akhtar, N.; Lam, S.-K.; and Ngo, T. 2025. Large language models for computer-aided design: A survey. *arXiv preprint arXiv:2505.08137*.

Zhong, X.; Liang, J.; and Li, Y. 2024. Building-agent: A 3d generation agent framework integrating large language models and graph-based 3d generation model. In Kontovourkis, O.; Phocas, M. C.; and Wurzer, G., eds., *eCAADe 2024*, eCAADe 2024: Data-Driven Intelligence - Proceedings Volume 2 of the 42nd Conference on Education and Research in Computer Aided Architectural Design in Europe, 291–300.